

Guia da Crocodrila prompta

Oct 1, 2026 · @Adriana Aneli Costa Lagrasta

O que você está recebendo

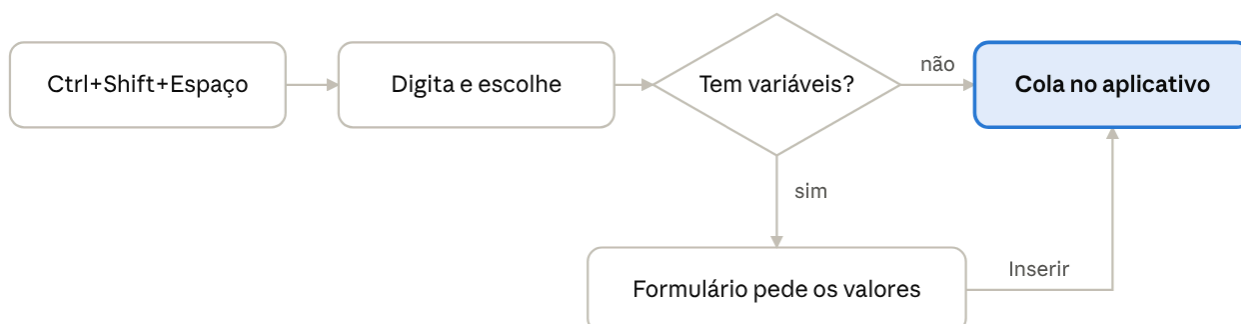
Você está recebendo uma caixa de ferramentas vazia e já montada: o programa que guarda, procura e cola prompts, sem nenhum prompt pronto além de três exemplos. Os prompts quem escreve é você.

Pense na Crocodrila como uma gaveta de modelos com um índice mágico. Em vez de abrir um documento, procurar o texto e copiar, você aperta um atalho, digita uma ou duas palavras e o prompt escolhido aparece colado onde o cursor estiver: no Claude, no ChatGPT, no Word, no navegador.

No dia a dia, o caminho é sempre o mesmo:

1. Você aperta `Ctrl+Shift+Espaço` e abre a paleta de busca.
2. Digita palavras do nome, da descrição ou das etiquetas do prompt.
3. Escolhe com as setas e aperta `Enter`.
4. Se o prompt tiver campos para preencher (as **variáveis**), uma janelinha pergunta os valores.
5. O texto final é colado no aplicativo que estava aberto.

Prompts com variáveis passam por um formulário antes de colar



uso da paleta · 4 passos, 1 desvio

Prompt sem variáveis vai direto para a colagem; com variáveis, o formulário entra no meio do caminho.

No ZIP vêm três tipos de coisa:

- **O motor:** os arquivos `.ahk`, que fazem a paleta funcionar. Você quase nunca vai mexer neles.

- **As ferramentas extras:** scripts em Python para gerar um catálogo em Word e para repetir um mesmo trecho em vários prompts.
- **Três prompts de exemplo**, para você ver como tudo funciona antes de criar os seus.

Antes de começar: o que instalar

Só uma coisa é obrigatória: o **AutoHotkey versão 2**, no Windows 10 ou 11. O Python é opcional e só serve para as ferramentas extras.

Programa	Precisa?	Para quê	Onde baixar
AutoHotkey v2	Sim	Faz a paleta, os atalhos e a colagem funcionarem	autohotkey.com
Python 3	Não	Catálogo em Word, bloco comum e testes	python.org
Git	Não	Guardar o histórico e enviar ao GitHub	git-scm.com

Atenção à versão do AutoHotkey. O site oferece a v1.1 e a v2. Escolha a **v2**. Os arquivos deste kit começam com a linha `#Requires AutoHotkey v2.0`, e a v1.1 não consegue abri-los.

Ao instalar o Python, marque a caixa **“Add Python to PATH”** na primeira tela. Sem ela, o comando `python` não é encontrado no terminal.

Instalação passo a passo

Há dois caminhos: o instalador automático, recomendado, ou quatro passos feitos à mão. Nada no código depende do lugar onde a pasta fica, então você pode guardá-la onde preferir.

Jeito rápido: o instalador

1. Clique com o botão direito no ZIP e escolha **“Extrair tudo”**. O instalador não funciona de dentro do ZIP fechado.
2. Na pasta extraída, dê dois cliques em **INSTALAR.cmd**. Se o Windows mostrar um aviso de segurança, clique em “Mais informações” e depois em “Executar assim mesmo”.
3. Uma janela preta mostra cada etapa. O instalador:
 - instala o AutoHotkey v2 pelo `winget`, o instalador de programas do Windows, se ele ainda não existir;
 - copia tudo para `C:\Users\SeuUsuario\crocodrila-prompta`;
 - cria o atalho que inicia a paleta com o Windows;

- pergunta se você quer instalar o Python (responda **s** ou **n**);
- abre a paleta e a pasta instalada.

Se a pasta `crocodrila-prompta` já existir, o instalador para sem copiar nada, para nunca apagar os seus prompts. Se o `winget` não estiver disponível, ele abre o site do AutoHotkey; instale a versão 2 e rode o `INSTALAR.cmd` de novo.

Jeito manual

Se preferir fazer tudo à mão, ou se o instalador falhar:

1. **Instale o AutoHotkey v2.** Na instalação, deixe marcada a opção que associa os arquivos `.ahk` ao programa. É ela que permite abrir um script com dois cliques.
2. **Descompacte o ZIP.** Clique com o botão direito em `crocodrila-prompta-kit.zip` e escolha “Extrair tudo”. Mova a pasta `crocodrila-prompta` para um lugar fixo, por exemplo `C:\Users\SeuUsuario\.prompt-library`. Evite deixá-la em Downloads, onde costuma ser apagada numa faxina.
3. **Rode a paleta.** Dê dois cliques em `prompt-library.ahk`. Nenhuma janela abre, e isso é normal: o programa fica escondido na bandeja, perto do relógio, com o ícone da crocodila. Se não enxergar o ícone, clique na setinha `^` da bandeja.
4. **Faça a paleta iniciar com o Windows (opcional, recomendado).** Aperte `Win+R`, digite `shell:startup` e aperte `Enter`. Na pasta que abrir, clique com o botão direito, escolha “Novo > Atalho” e aponte para o arquivo `watchdog.ahk` da sua pasta.

O `watchdog.ahk` é o **vigia**. A cada 60 segundos ele confere se a paleta está rodando e, se não estiver, abre de novo. Por isso o atalho de inicialização aponta para ele, e não para a paleta: ele cuida do resto.

Primeiro voo: testando com os exemplos

Antes de escrever qualquer prompt, faça este teste de cinco minutos. Se ele funcionar, a instalação está certa.

1. Abra o Bloco de Notas e deixe o cursor numa linha vazia.
2. Aperte `Ctrl+Shift+Espaço`. A paleta escura aparece por cima de tudo.
3. Digite `exemplo`. Aparecem os três prompts de demonstração.
4. Escolha “Exemplo - prompt simples” e aperte `Enter`. O texto é colado no Bloco de Notas.
5. Repita com “Exemplo - prompt com variáveis”. Agora surge um formulário com os campos **Tema** e **Público-alvo**. Preencha e clique em **Inserir**. Veja que as palavras entre chaves, como `{tema}`, foram trocadas pelo que você digitou.

Atalhos que valem a pena decorar:

Tecla	O que faz
Ctrl+Shift+Espaço	Abre a paleta
Setas ↑↓	Andam pela lista
Enter ou duplo clique	Escolhe o prompt
Esc	Fecha a paleta ou o formulário sem colar nada
Ctrl+Shift+F5	Recarrega os prompts depois de criar ou editar um arquivo

Um detalhe útil: a paleta usa a área de transferência só por um instante. Logo depois de colar, ela devolve o que você tinha copiado antes. Você não perde o seu Ctrl+C.

Anatomia de um prompt

Cada prompt é um arquivo de texto com extensão `.yaml`, que funciona como uma ficha: alguns campos de cabeçalho e, no fim, o texto do prompt. Este é o exemplo com variáveis que veio no kit:

```
name: Exemplo - prompt com variáveis
description: Abre um formulário antes de colar; cada {variável} vira um campo
tags: [exemplo, variaveis]
variables:
  - name: tema
    label: Tema
    placeholder: "ex: fotossíntese"
  - name: publico
    label: Público-alvo
    placeholder: "ex: alunos do 9o ano"
    default: "leigos"
prompt: |
  Explique {tema} para {publico}.
  Use no máximo três parágrafos e um exemplo do cotidiano.
```

Campo por campo:

Campo	Obrigatório?	O que é
name	Sim	O nome que aparece na lista da paleta

Campo	Obrigatório?	O que é
<code>description</code>	Não	Uma linha explicando o que o prompt faz; também entra na busca
<code>tags</code>	Não	Etiquetas para achar o prompt, sempre entre colchetes e separadas por vírgula
<code>variables</code>	Não	A lista de campos que o formulário vai pedir
<code>prompt</code>	Sim	O texto que será colado

Cada variável tem até quatro partes:

- `name` : o nome técnico, sem espaço nem acento. É ele que vai entre chaves no texto: `{publico}`.
- `label` : o rótulo que aparece em cima do campo no formulário.
- `default` : um valor que já vem preenchido e que você pode trocar.
- `placeholder` : uma dica para quem edita o arquivo. Nesta versão, a janela não mostra essa dica. Se quiser algo visível no campo, use `default`.

As três regras que evitam 90% dos problemas

1. **Recuo de dois espaços.** Todas as linhas do texto, abaixo de `prompt: |`, começam com dois espaços. O texto termina na primeira linha que encosta na margem. Linhas em branco no meio do texto podem ficar.
2. **Chaves batendo com nomes.** Toda `{palavra}` usada no texto precisa existir em `variables` com o mesmo `name`, escrito igualzinho. Uma chave sem variável correspondente é colada como está, com as chaves.
3. **Salvar em UTF-8.** No Bloco de Notas, use “Salvar como” e confira a codificação no canto inferior da janela. Sem UTF-8, os acentos viram símbolos estranhos.

Use espaços, nunca a tecla `Tab`, para recuar. Um editor como o [VS Code](#) ajuda: ele mostra o recuo e salva em UTF-8 por padrão.

Organizando a sua biblioteca

A organização é feita só com pastas, sem cadastro em lugar nenhum. A primeira pasta dentro de `prompts/` é a **área**; a segunda é a **tarefa**. Ambas entram na busca.

```

prompts/
  faculdade/                ← área
    resumos/                ← tarefa
      resumo-capitulo.yaml
      resumo-artigo.yaml
    provas/
      simulado-questoes.yaml
  trabalho/
    emails/
      resposta-cliente.yaml

```

As áreas e tarefas acima são só ilustração; crie as que fizerem sentido para a sua vida.

Algumas convenções que facilitam a vida:

- **Nomes de pasta e de arquivo** em minúsculas, sem acento e com hífen no lugar do espaço: `resumo-capitulo.yaml`. O nome bonito, com acento, vai no campo `name`.
- **Dois níveis bastam.** A busca só enxerga a área e a tarefa; uma terceira pasta funciona, mas o nome dela não é pesquisado.
- **Tags pensando na busca.** A paleta só mostra um prompt se *todas* as palavras digitadas aparecerem no nome, na descrição, na área, na tarefa ou nas tags. E ela diferencia acento: quem digita `analise` não acha “Análise”. Vale colocar nas tags a versão sem acento das palavras que você costuma digitar.
- **Um prompt por arquivo.** Se o arquivo não tiver `name` ou `prompt`, a paleta simplesmente o ignora, sem aviso.

Depois de testar, apague os exemplos: remova a pasta `prompts/exemplos/` e a linha correspondente do arquivo `padroes/alvos.txt`. Os testes automáticos da pasta `tests/` usam esses exemplos e vão falhar sem eles. Se você não for rodar testes, pode ignorá-los.

Usando o Claude Code a seu favor

A pasta já vem com um arquivo `CLAUDE.md`, que funciona como um manual de instruções para o Claude. Quando você abre o Claude Code dentro da pasta, ele lê esse arquivo e passa a saber onde ficam os prompts, qual é o formato exato do YAML e quais regras não pode quebrar.

Para começar, abra o terminal dentro da pasta. No Explorador de Arquivos, clique na barra de endereço, digite `cmd` e aperte `Enter`. Depois digite `claude` e aperte `Enter`.

Pedidos que funcionam bem:

- “Crie um prompt em `prompts/faculdade/resumos/` que resuma um capítulo de livro em tópicos, com uma variável para o número de tópicos.”

- “Transforme este texto que eu uso sempre num prompt da biblioteca:” e cole o texto em seguida.
- “Confira se todos os prompts estão no formato certo: recuo, variáveis declaradas, UTF-8.”
- “Sugira tags melhores para os prompts da pasta `trabalho`, pensando no que eu digitaria para achá-los.”
- “O prompt X não aparece na paleta. Descubra por quê.”

Depois de qualquer criação ou edição, aperte `Ctrl+Shift+F5` para a paleta recarregar.

O `CLAUDE.md` é seu. Se você tiver preferências, como o idioma das respostas, um tom de escrita ou uma regra do tipo “nunca apague nada sem me perguntar”, acrescente-as lá em linguagem comum. O Claude passa a segui-las em toda sessão aberta nesta pasta.

Recursos extras

Nada nesta seção é necessário para usar a paleta. São ferramentas para quando a biblioteca crescer. As três primeiras usam Python; antes delas, rode uma vez no terminal:

```
pip install python-docx pyyaml pytest.
```

Bloco comum: escreva uma vez, use em vários prompts

Com o tempo aparecem instruções que você repete em muitos prompts, como “responda em português” ou “não invente fontes”. Em vez de copiar e colar, e depois ter de corrigir em dez lugares, você as escreve num só arquivo.

1. Escreva as regras em `padroes/bloco-comum.txt`, mantendo a primeira linha (`=== BLOCO COMUM ...`) e a última (`=== FIM DO BLOCO COMUM ===`).
2. Liste em `padroes/alvos.txt` os prompts que devem receber o bloco, um caminho por linha.
3. Rode `python sincronizar_padrao.py`. O bloco é colado no fim de cada prompt listado ou, se já estiver lá, atualizado.

Não edite o bloco dentro dos prompts: a próxima sincronização desfaz a edição. Para só conferir, sem gravar nada, rode `python sincronizar_padrao.py --verificar`. O exemplo `prompt-com-bloco-comum.yaml` mostra o resultado.

Catálogo em Word

`python gerar_catalogo.py` cria o arquivo `catalogo-prompts.docx`, uma tabela com todos os prompts, organizados por área e tarefa. Serve para revisar a coleção ou mostrá-la a alguém.

Testes

`pytest` confere se o sincronizador continua funcionando. Os arquivos `test-yaml-parser.ahk` e `test-prompt-loader.ahk`, na pasta `tests/`, testam o motor da paleta: dê dois cliques e uma janela mostra o resultado.

Histórico no Git e cópia no GitHub

O Git guarda cada versão dos seus prompts, e você pode voltar atrás se estragar algum. Para começar, rode na pasta:

```
git init
git add -A
git commit -m "Primeira versao"
```

Se quiser uma cópia na nuvem, crie um repositório **privado** no GitHub e conecte-o seguindo as instruções da própria página. Daí em diante, dois cliques em `subir-mudancas.bat` registram e enviam as alterações. Se não souber por onde começar, peça ao Claude Code: “me ajude a colocar esta pasta num repositório privado do GitHub”.

Personalizando atalhos, aparência e nome

- **Atalhos e janela:** edite `config.yaml`. No nome das teclas, `^` é Ctrl, `+` é Shift, `!` é Alt e `#` é a tecla Windows. Por exemplo, `"^!p"` seria Ctrl+Alt+P. As opções `window_width` e `max_results` controlam a largura da paleta e quantos resultados ela mostra.
- **Ícones:** troque os arquivos da pasta `assets/`, mantendo os mesmos nomes. O ícone da bandeja precisa ser `.ico`.
- **Nome:** “Crocodrila prompta” aparece no título da paleta (`lib/SearchGui.ahk`) e na dica da bandeja (`lib/TrayMenu.ahk`). Se quiser batizar a sua, peça ao Claude Code para trocar.

Quando algo dá errado

Quase todo problema está num destes casos. Quando nenhum servir, abra o Claude Code na pasta e descreva o que acontece.

O que você vê	Causa provável	O que fazer
Mensagem dizendo que o script exige AutoHotkey v2	A versão instalada é a 1.1	Instale a v2 pelo site oficial

O que você vê	Causa provável	O que fazer
O atalho não abre nada	A paleta não está rodando, ou outro programa usa a mesma combinação	Procure a crocodila na bandeja; se ela estiver lá, troque o atalho em <code>config.yaml</code>
Um prompt novo não aparece	Falta recarregar, ou falta <code>name</code> ou <code>prompt</code>	Aperte <code>Ctrl+Shift+F5</code> ; confira os dois campos
O prompt aparece cortado	Alguma linha do texto ficou sem os dois espaços de recuo	Recue a linha onde o texto parou
Acentos viram símbolos estranhos	O arquivo não foi salvo em UTF-8	Salve de novo, escolhendo UTF-8
Sai <code>{tema}</code> literalmente no texto colado	O nome entre chaves não bate com o <code>name</code> da variável	Corrija a grafia em um dos dois lugares
A busca não acha um prompt que existe	Uma das palavras digitadas não está no nome, na descrição nem nas tags, ou o acento é diferente	Digite menos palavras, ou acrescente tags
O texto não é colado num aplicativo específico	Alguns programas bloqueiam o <code>Ctrl+V</code> simulado	Clique no campo de texto antes de abrir a paleta; se persistir, copie o texto direto do arquivo <code>.yaml</code>
<code>python</code> não é reconhecido	O Python não está no PATH	Reinstale o Python marcando “Add Python to PATH”

Mapa da pasta e glossário

No uso diário, você só mexe em `prompts/`, às vezes em `config.yaml` e em `padroes/`. O resto é motor. Os arquivos `INSTALAR.cmd` e `instalar.ps1` vêm só no ZIP e servem apenas para a primeira instalação.

Arquivo ou pasta	Para que serve	Você mexe?
<code>prompts/</code>	Os seus prompts, em arquivos <code>.yaml</code>	Sempre

Arquivo ou pasta	Para que serve	Você mexe?
<code>config.yaml</code>	Atalhos, largura da janela, número de resultados	Às vezes
<code>padroes/</code>	O bloco comum e a lista de prompts que o recebem	Às vezes
<code>CLAUDE.md</code>	Instruções para o Claude Code	Quando quiser ajustar
<code>LEIA-ME-PRIMEIRO.md</code> e <code>README.md</code>	Resumo técnico da instalação e da estrutura	Não
<code>prompt-library.ahk</code>	O programa principal: é ele que você abre	Não
<code>watchdog.ahk</code>	O vigia que reabre a paleta	Não
<code>lib/</code>	As peças do motor: busca, formulário, colagem, leitura do YAML	Não
<code>assets/</code>	Ícones	Só para trocar a aparência
<code>gerar_catalogo.py</code>	Gera o catálogo em Word	Não, só roda
<code>sincronizar_padrao.py</code>	Aplica o bloco comum	Não, só roda
<code>subir-mudancas.bat</code>	Envia as mudanças ao GitHub	Não, só roda
<code>tests/</code>	Testes automáticos	Não
<code>.gitignore</code>	Diz ao Git o que não guardar	Não

Palavras que aparecem neste guia:

- **Prompt:** o texto de instrução que você entrega a uma IA.
- **Paleta:** a janela de busca que abre com o atalho.
- **YAML:** o formato de ficha dos arquivos de prompt, com campos no formato `nome: valor`.
- **Variável:** um espaço em branco no prompt, escrito `{assim}`, preenchido na hora de colar.
- **Recuo:** os espaços no começo da linha.
- **UTF-8:** o jeito de salvar texto que preserva os acentos.
- **Bandeja:** a área de ícones pequenos ao lado do relógio do Windows.

- **Script:** um arquivo com instruções que o computador executa, como os `.ahk` e os `.py`.
- **Terminal:** a janela preta onde se digitam comandos como `python` e `claude`.